AI-POWERED E-COMMERCE ANALYTICS AND CONVERSATIONAL AGENT SYSTEN

Data Collection, Preprocessing and EDA Report

Course: Artificial Intelligence

Team members:

Eman Fatima(24L-3008)

Rabia(24L-3093)

Abdullah Tahir(24L-3061)

Shahzaib Zia(24L-3057)

Notebook file: F26-07.ipynb

1. Dataset source and description

Our project is an AI agent that talks directly to customers of small fashion shops that sell over WhatsApp and Instagram. Customers write in a mix of Roman Urdu and English, for example "black kurta medium mein hai?", so the agent has to understand the message and then do the right thing: check stock, take an order, look up an order, handle a simple exchange, or pass the chat to a human when it is not sure. We could not find a public dataset that fits this, so we built our own. All of it is made up, including the names, phone numbers and addresses, so no real personal data is used.

Part	What it contains	Size
Product catalog (catalog.csv)	Product variants (product, color, size) with original price, selling price, stock, a sale flag and a short description. This is the source of truth for price and stock.	572 rows, 50 products, 16 categories, 31 colors, 25 sizes. Prices from Rs. 699 to 9,499 (median 2,499).
Store policy (policy.md, policy_rules.json)	Rules of a made-up store: delivery charges by city, COD limit, order limits, 7-day exchange rules, refund rules and when to hand over to a human. The JSON file has the same rules in a form the code can read.	About 1,300 words in 10 sections, and 8 rule groups.
Orders (orders.csv)	A small order history (Placed, Packed, Shipped, Delivered). We need it for the order status and exchange conversations.	51 orders
Conversations (conversations.json)	Customer conversations with labels: expected intent, entities, expected action, whether it should be escalated and why, expected result, difficulty tags and a dev/test split.	172 conversations, 8 categories, 357 customer messages

We looked at MultiWOZ but decided not to use it. It covers hotel, restaurant, train and taxi booking in English only, and it has no product, size, color or quantity fields, so it does not match an order-taking task.

2. Data collection method

- **Catalog.** A Python script (generate_catalog.py) creates 50 fashion products with colors and sizes. We chose the prices ourselves in a realistic range and did not scrape any store. Stock is random: about 10% of variants are set to zero and about 20% are kept low. Sale items get 20% off and keep their original price. A few rows are fixed by hand so that our example conversations match the catalog.
- **Policy and orders.** We wrote the policy ourselves for a made-up shop. The orders are generated by a script so that totals follow the catalog prices and the delivery rules (Rs. 250 or Rs. 350, free above Rs. 10,000).
- **Conversations.** 20 conversations were written by hand and the other 152 were generated from templates that use the catalog, orders and policy, so the facts inside them are always correct. Every label (totals, stock, exchange eligibility, escalation reason) is calculated from the data and not typed in. The templates mix Roman Urdu and English and add spelling variants such as blk, kala and mdm, different phone number formats and references to earlier messages.
- **Checking.** A separate script (validate_dataset.py) recalculates every label that can be checked and compares it with the dataset. It found no errors. The generated conversations still need a manual read-through to make sure the Roman Urdu sounds natural.
- **Split.** 70/30 inside each category, which gives 120 development and 52 test conversations. We will not tune prompts on the test conversations.

3. Preprocessing steps

Step	What we did	Result
Quality checks	Checked data types, missing values, duplicate SKUs and variants, negative stock and price logic (sale price must be below the original price).	No problems found, which is expected because we generated the catalog. The checks are written as functions so they can be re-run.
Types and text	Kept size as text because it mixes M, 32, 2-3Y and Free. Made is_sale a true/false column, fixed color casing and removed extra spaces.	Stops a size like 30 from turning into a number and breaking matching.
New columns	Added discount_pct, stock_status (in stock, low stock under 5, out of stock) and a variant key.	catalog_clean.csv has 13 columns.
Roman Urdu text cleaning	Lowercase, removed extra spaces, line breaks and emojis, and made phone numbers look the same.	"Mera naam Ali hai, number 0300-1234567" becomes "mera naam ali hai, number 03001234567". 117 of 172 conversations changed.
Synonyms and fuzzy matching	Built a table of aliases for colors (kala, safaid, gulabi), sizes (mdm, medium) and products. rapidfuzz maps misspelled words such as blk, blak and marun to the catalog value.	69 entries in synonyms.csv. Used to check and clean entities against the catalog.
Conversation checks	Checked that every action is valid, the escalation flag matches the action, every item exists in the catalog, every order ID exists, and there are no empty or duplicate conversations.	0 label problems and 0 duplicates.
Policy chunking for RAG	Split the policy by section heading into chunks of at most 250 words, keeping the heading with each chunk.	10 chunks, 121 words on average (policy_chunks.json).

We also tested whether the cleaning plus fuzzy matching can find the labeled color and size in the message text. It found almost all of them, but this number does **not** show real accuracy. The conversations were generated from the same alias list, so the test is partly circular. It only tells us that the data and the pipeline are consistent. We will measure real accuracy later on the test split.

4. Brief EDA (key insights)

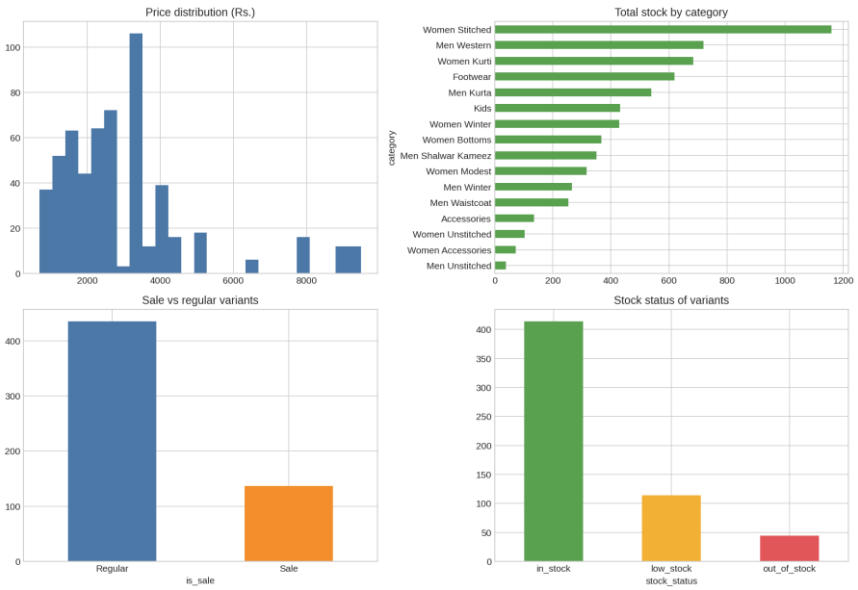


Figure 1. Catalog. About 8% of variants are out of stock and 20% are low on stock (under 5 units). So stock checks will often fail or hit a limit, which is what we want for the out-of-stock and concurrent order tests. 24% of variants are on sale and cannot be exchanged. Women Stitched is the biggest category and unstitched fabric the smallest. We kept this imbalance because real shops look like that.

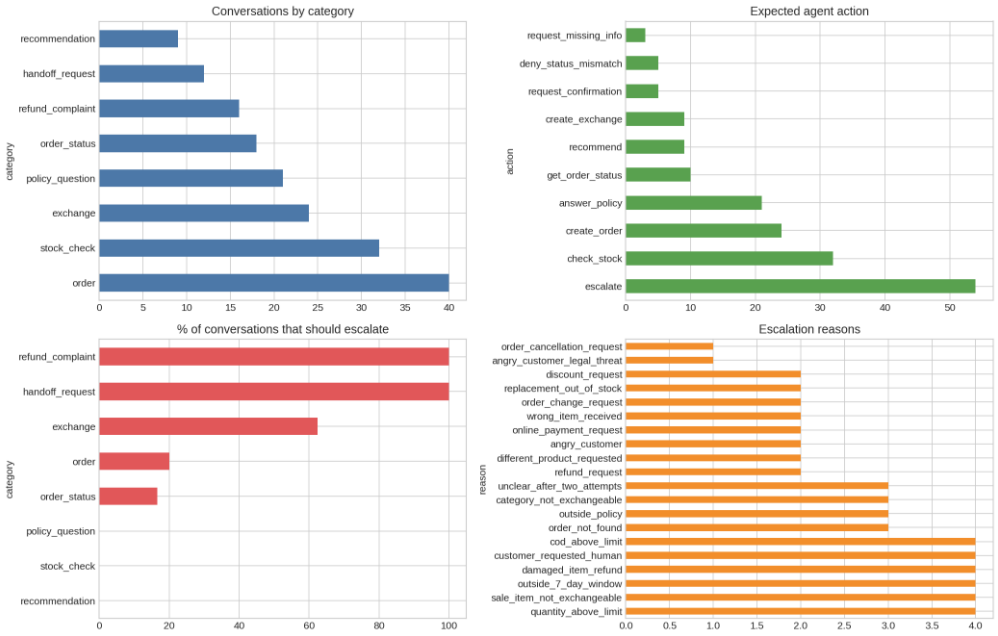


Figure 2. Conversations. Order (40), stock check (32) and exchange (24) are the biggest categories. Recommendation is the smallest (9), so results there will be noisy. 31% of conversations should end in escalation (35% in the test split), so a system that never escalates can score at most 69% on action. Each escalation reason has only 1 to 4 examples, which is too few to make claims about individual reasons.

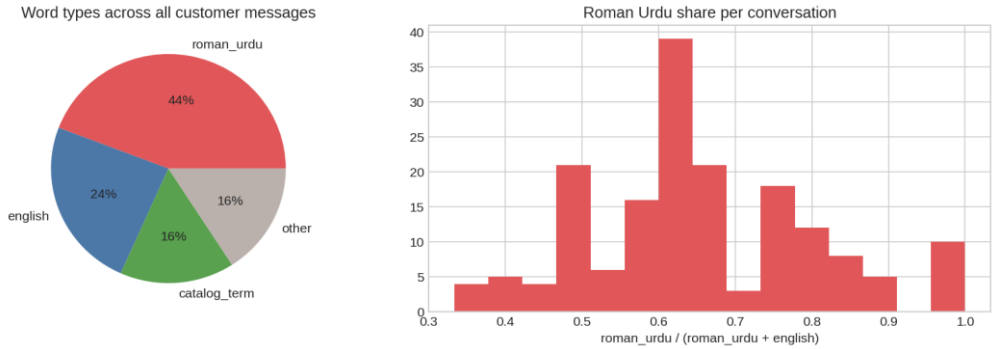


Figure 3. Code-mixing. 94% of conversations contain both Roman Urdu and English words, and about 65% of the language words (not counting product terms) are Roman Urdu. The language understanding part therefore has to handle both inside one message. This comes from a simple word-list method, so treat it as an estimate. Other numbers: 2.1 customer messages and 13 words per conversation on average, 52% are multi-turn, 63% are tagged as hard cases (phone formats, spelling variants, references to earlier messages), and 41 of the 50 products appear in conversations.

5. Final dataset summary

File	Contents	Rows	Used for
catalog_clean.csv	Product variants	572 (13 columns)	Price and stock through tool calls
orders_clean.csv	Mock orders	51 (12 columns)	Order status and exchange checks
policy_chunks.json / policy_rules.json	Policy text chunks / rules	10 chunks / 8 groups	RAG / guardrail layer
conversations_clean.json	Labeled conversations	172 (120 dev, 52 test)	Development and evaluation
synonyms.csv	Color, size and product aliases	69	Cleaning and checking entities

Limitations

- All data is synthetic. Prices, stock and the policy belong to a made-up store.

- 152 of the 172 conversations come from templates, so the language is cleaner and less varied than real WhatsApp chats. We will add conversations written by hand, mainly to the test split.
- The set is small, so results per category or per escalation reason will be uncertain.
- Customer messages are scripted, so testing long multi-turn chats will need a flexible test harness or a simulated customer.
- There are only a few adversarial cases (attempts to break the rules), and we plan to add more.